

UFU/FACOM/BCC

Disciplina: Algoritmos e Estruturas de Dados II

Ref: Atividade Prática relacionada a algoritmos de ordenação

1. Objetivos

- Implementar algoritmos clássicos de ordenação.
- Analisar complexidade computacional (matematicamente e empiricamente)
- Desenvolver raciocínio sobre estabilidade e eficiência.

2. Fundamentos Teóricos

Algoritmos de ordenação são amplamente utilizados para organizar dados em aplicações computacionais. Diferentes algoritmos possuem estratégias distintas de ordenação e custos computacionais variados.

Nesta prática serão analisados os algoritmos Bubble Sort, Selection Sort, Insertion Sort, QuickSort e MergeSort.

3. Atividades

3.1 Bubble Sort. Complete o código abaixo.

```
void bubbleSort(int v[], int n){  
    int i, j, temp;  
    for(i = 0; i < n-1; i++){  
        for(j = 0; j < _____; j++){  
            if(v[j] > v[j+1]){  
                temp = v[j];  
                v[j] = _____;  
                v[j+1] = temp;  
            }  
        }  
    }  
}
```

Perguntas:

- a) Qual a complexidade do pior caso?

- b) O Bubble Sort é estável?
- c) Modifique o código considerando que o array possa estar ordenado.

3.2. Selection Sort. Complete o código abaixo.

```
void selectionSort(int v[], int n){
    int i, j, min, temp;
    for(i = 0; i < n-1; i++){
        min = _____;
        for(j = i+1; j < n; j++){
            if(v[j] < v[min]){
                min = _____;
            }
        }
        temp = v[i];
        v[i] = v[min];
        v[min] = temp;
    }
}
```

Perguntas:

- a) Qual a ideia principal do Selection Sort?
- b) Quantas trocas aproximadamente são realizadas?
- c) O algoritmo é estável?

3.3. Insertion Sort. Complete o código abaixo.

```
void insertionSort(int v[], int n){
    int i, j, chave;
    for(i = 1; i < n; i++){
        chave = v[i];
        j = i - 1;
        while(j >= 0 && v[j] > chave){
            v[j+1] = _____;
            j--;
        }
        v[j+1] = _____;
    }
}
```

Perguntas:

- a) Em quais situações o Insertion Sort é eficiente?
- b) Qual o melhor caso do algoritmo?
- c) O algoritmo é adaptativo?

3.4 QuickSort. Complete o código abaixo.

```
int particiona(int v[], int inicio, int fim){
    int pivo = v[fim];
    int i = inicio - 1;
    int j, temp;
    for(j = inicio; j < fim; j++){
        if(v[j] <= _____){
            i++;
            temp = v[i];
            v[i] = v[j];
            v[j] = temp;
        }
    }
    temp = v[i+1];
    v[i+1] = v[fim];
    v[fim] = temp;
    return _____;
}
```

```
void quickSort(int v[], int inicio, int fim){
    if(inicio < fim){
        int p = particiona(v, inicio, fim);
        quickSort(v, inicio, _____);
        quickSort(v, _____, fim);
    }
}
```

Perguntas:

- a) O que é o pivô?
- b) Quando ocorre o pior caso do QuickSort?
- c) Qual a complexidade média do algoritmo?
- d) Prove matematicamente que a complexidade do algoritmo é $O(n \log n)$,

3.5. MergeSort. Complete o código abaixo.

```
void merge(int v[], int inicio, int meio, int fim){
    int i = inicio;
    int j = meio + 1;
    int k = 0;
    int temp[1000];
    while(i <= meio && j <= fim){
        if(v[i] < v[j]){
            temp[k++] = _____;
        }else{
            temp[k++] = _____;
        }
    }
    while(i <= meio){
        temp[k++] = v[i++];
    }
    while(j <= fim){
        temp[k++] = v[j++];
    }
    for(i = inicio, k = 0; i <= fim; i++, k++){
        v[i] = _____;
    }
}
```

Perguntas:

- a) Explique a estratégia dividir para conquistar.
- b) Qual a principal vantagem do MergeSort?
- c) Qual sua principal desvantagem?

3.6. Atividade Experimental

Implemente um programa que execute os algoritmos para diferentes tamanhos de vetor (1000, 5000 e 10000 elementos).

Utilize a função `clock()` para medir os tempos.

Perguntas:

- Qual algoritmo apresentou melhor desempenho?
- Os resultados confirmam a teoria?
- Qual algoritmo apresentou maior crescimento de tempo?

4. Conclusão

Apresente uma análise comparativa entre os algoritmos estudados.

[illegible]